

Specialized Monitoring Scenarios

Series: K8S | **Notebook:** 12 of 12 | **Created:** January 2026 | **Last Updated:** 01/30/2026

NGINX Ingress, CSI Driver, and Resource Tuning

This notebook covers specialized monitoring scenarios including NGINX Ingress Controller instrumentation, CSI Driver architecture, and resource sizing guidelines for Dynatrace components.

Table of Contents

- [1. NGINX Ingress Controller Monitoring](#)
 - [2. CSI Driver Architecture](#)
 - [3. CSI Driver Resource Configuration](#)
 - [4. Component Sizing Guidelines](#)
 - [5. Telemetry Ingest Configuration](#)
 - [6. Troubleshooting Specialized Scenarios](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring
Kubernetes Cluster	Dynatrace Operator v1.0+ installed
NGINX Ingress	ingress-nginx controller (for Section 2)
Knowledge	Completed K8S-01 through K8S-02

1. NGINX Ingress Controller Monitoring

Why Monitor NGINX Ingress?

The NGINX Ingress Controller is often the entry point for all external traffic. Instrumenting it provides:

Benefit	Description
End-to-end traces	Complete request path visibility
Ingress latency	Measure time spent in ingress layer
Error correlation	Link 5xx errors to backend services
Traffic analysis	Understand traffic patterns

Prerequisites for NGINX Instrumentation

Requirement	Details
OneAgent version	1.227+
Pod naming	Must contain <code>ingress-nginx-</code>
Architecture	x86-64 only
Controller type	ingress-nginx (not nginx-ingress)

Configuration Steps

Step 1: Edit the ingress-nginx-controller ConfigMap

```
kubectl edit configmap ingress-nginx-controller -n ingress-nginx
```

Step 2: Add the OneAgent module loading snippet

```
data:
  main-snippet: |
    load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-
x86-64/liboneagentnginx.so;
```

Step 3 (Optional): Add trace context to access logs

```
data:
  main-snippet: |
    load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-
x86-64/liboneagentnginx.so;
    log-format-upstream: &gt;-
      $remote_addr - $remote_user [$time_local] "$request"
      [!dt
dt.trace_id=$dt_trace_id,dt.span_id=$dt_span_id,dt.trace_sampled=$dt_trace_sa
mpled]
      $status $body_bytes_sent "$http_referer" "$http_user_agent"
```

Step 4: Rolling restart the ingress controller

```
kubectl rollout restart deployment ingress-nginx-controller -n ingress-nginx

# Watch rollout
kubectl rollout status deployment ingress-nginx-controller -n ingress-nginx
```

Verifying NGINX Instrumentation

```
# Check OneAgent injection
kubectl -n ingress-nginx exec -it deploy/ingress-nginx-controller -- cat
/proc/1/maps | grep dynatrace

# Check for errors in logs
kubectl -n ingress-nginx logs -l app.kubernetes.io/name=ingress-nginx | grep
-i dynatrace
```

Note: No DynaKube changes are required for NGINX monitoring - only the ConfigMap edit and pod restart.

```
// Check if NGINX Ingress spans are arriving
fetch spans
| filter contains(service.name, "nginx") or contains(service.name, "ingress")
| summarize count = count(), by:{service.name, span.name}
| sort count desc
| limit 20
```

2. CSI Driver Architecture

Understanding the CSI Driver

The CSI (Container Storage Interface) Driver provides OneAgent code modules to application pods via volumes instead of host mounts.

CSI Driver Container Structure

The CSI Driver DaemonSet runs **5 containers** in a sidecar pattern:

Container	Purpose	Resource Impact
csi-init	Initialize volume plugin	Runs once at startup
server	Main CSI plugin	Core functionality
provisioner	Volume provisioning	Highest resource needs
registrar	Node registration	Low overhead
livenessprobe	Health checking	Minimal

3. CSI Driver Resource Configuration

Helm Values for CSI Driver

Configure resources for each CSI Driver container in your Helm `values.yaml` :

```
csidriver:
  enabled: true

# Init container – runs once at startup
csiInit:
  resources:
    requests:
      cpu: 50m
      memory: 100Mi
    limits:
      cpu: 100m
      memory: 128Mi

# Main CSI plugin
server:
  resources:
    requests:
      cpu: 50m
      memory: 100Mi
    limits:
      cpu: 100m
      memory: 128Mi

# Volume provisioner – needs more resources
provisioner:
  resources:
    requests:
      cpu: 300m
      memory: 100Mi
    limits:
      cpu: 500m          # Often missing in defaults!
      memory: 256Mi      # Often missing in defaults!

# Node registration sidecar
registrar:
  resources:
    requests:
      cpu: 20m
      memory: 30Mi
    limits:
      cpu: 50m
      memory: 64Mi

# Health check sidecar
livenessprobe:
  resources:
    requests:
```

```
cpu: 20m
memory: 30Mi
limits:
  cpu: 50m
  memory: 64Mi
```

CSI Driver Resource Summary Table

Container	CPU Request	CPU Limit	Memory Request	Memory Limit	Notes
csiInit	50m	100m	100Mi	128Mi	Init container
server	50m	100m	100Mi	128Mi	Main plugin
provisioner	300m	500m	100Mi	256Mi	Add limits!
registrar	20m	50m	30Mi	64Mi	Sidecar
livenessprobe	20m	50m	30Mi	64Mi	Sidecar

Warning: Default `values.yaml` may be missing limits for the `provisioner` container. Add them to ensure predictable resource usage.

4. Component Sizing Guidelines

ActiveGate Sizing

Cluster Size	Nodes	CPU Limit	Memory Limit	Replicas
Small	1-10	500m	1Gi	1
Medium	10-50	1000m	2Gi	2
Large	50-100	2000m	4Gi	2
Enterprise	100+	4000m	8Gi	3+

OneAgent Resources

Mode	CPU Request	CPU Limit	Memory Request	Memory Limit
cloudNativeFullStack	100m	300m	256Mi	512Mi
classicFullStack	150m	500m	512Mi	1Gi
applicationMonitoring	50m	200m	128Mi	256Mi

OTel Collector Sizing

Telemetry Volume	CPU Limit	Memory Limit	Use Case
Low	200m	256Mi	Dev/Test
Medium	500m	512Mi	Staging
High	1000m	1Gi	Production
Very High	2000m	2Gi	High-throughput

Log Monitoring Sizing

Log Volume	CPU Limit	Memory Limit	Notes
Low	200m	256Mi	< 1GB/day
Medium	500m	512Mi	1-10GB/day
High	1000m	1Gi	10-50GB/day
Very High	2000m	2Gi	50GB+/day

5. Telemetry Ingest Configuration

Multi-Protocol Ingestion

Configure DynaKube to accept telemetry from multiple sources:


```
spec:
  telemetryIngest:
    protocols:
      - otlp      # OpenTelemetry Protocol
      - jaeger    # Jaeger traces
      - zipkin    # Zipkin traces
      - statsd    # StatsD metrics
    serviceName: telemetry-ingest
```

Protocol Endpoints

Protocol	Default Port	Endpoint
OTLP gRPC	4317	telemetry-ingest.dynatrace:4317
OTLP HTTP	4318	http://telemetry-ingest.dynatrace:4318
Jaeger	14268	http://telemetry-ingest.dynatrace:14268
Zipkin	9411	http://telemetry-ingest.dynatrace:9411
StatsD	8125	telemetry-ingest.dynatrace:8125 (UDP)

Application Configuration

```
# Application deployment using OTLP
env:
  - name: OTEL_EXPORTER_OTLP_ENDPOINT
    value: "http://telemetry-ingest.dynatrace:4318"
  - name: OTEL_SERVICE_NAME
    value: "my-application"
```

```
// Check CSI driver pod status
fetch logs
| filter matchesPhrase(content, "csi") and matchesPhrase(content,
"dynatrace")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

```
// Monitor telemetry ingest volume by protocol
fetch spans
| filter isNotNull(otel.scope.name)
| summarize span_count = count(), by:{service.name}
| sort span_count desc
| limit 15
```

6. Troubleshooting Specialized Scenarios

NGINX Ingress Issues

Issue	Cause	Solution
Module load fails	Wrong architecture	x86-64 only, no ARM
No spans from ingress	Pod name mismatch	Must contain <code>ingress-nginx-</code>
Partial traces	OneAgent version	Upgrade to 1.227+

```
# Debug NGINX module loading
kubectl -n ingress-nginx exec -it deploy/ingress-nginx-controller -- \
  ls -la /opt/dynatrace/oneagent-paas/agent/bin/current/
```

CSI Driver Issues

Issue	Cause	Solution
Volume mount fails	Provisioner OOM	Increase provisioner limits
Slow pod startup	CSI driver overloaded	Add resources, check node count
Code modules missing	CSI not enabled	Enable in Helm values

```
# Check CSI driver status
kubectl -n dynatrace get pods -l app.kubernetes.io/component=csi-driver

# View CSI driver logs
kubectl -n dynatrace logs -l app.kubernetes.io/component=csi-driver -c server
```

Resource Exhaustion

Symptom	Component	Action
OOMKilled	Any	Increase memory limits
CPU throttling	Any	Increase CPU limits
Slow queries	ActiveGate	Add replicas or resources
Dropped spans	OTel Collector	Increase batch size, resources

Summary

In this notebook, you learned:

- **NGINX Ingress monitoring** with OneAgent module loading
- **CSI Driver architecture** and the 5-container structure
- **CSI Driver resource configuration** with per-container limits
- **Component sizing guidelines** for all Dynatrace components
- **Telemetry ingest configuration** for multi-protocol support
- **Troubleshooting** common specialized monitoring issues

References

- [NGINX Instrumentation](#)
- [CSI Driver Configuration](#)
- [Dynatrace Operator Helm Chart](#)
- [Telemetry Ingest](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.